4.7.1. Introducere în criptografie

Criptosistem. Metoda fundamentală de securizare a comunicaţiei între două părţi A şi B faţă de o terţă parte C constă în stabilirea, de comun acord de către A şi B, a unui *sistem de criptare* (*cifrare*) a mesajelor. Un astfel de sistem trebuie să ofere posibilitatea criptării mesajelor ce urmează a fi transmise şi, totodată, a decriptării "corecte" a mesajelor (criptate) recepţionate. De exemplu, A şi B pot conveni ca fiecare literă a mesajului să fie înlocuită ciclic cu litera aflată la distanţa k de litera în cauză (considerând alfabetul ordonat în mod uzual). Astfel, pentru $k = 2$, litera a s-ar înlocui cu c, litera b cu d etc. Vom spune în acest caz că *cheia de criptare* este $k = 2$; ea poate fi schimbată la fiecare nouă comunicare (cu condiţia de a putea fi stabilită de A şi B în deplină securitate).

Mesajele originale, ce urmează a fi criptate şi transmise, se construiesc pe baza unei mulţimi finite de elemente "atomice", mulţime numită *alfabet*. Elementele acestei mulţimi pot fi literele şi cifrele caracteristice unei anumite limbi (engleză, japoneză, chineză etc.), secvenţe formate din anumite litere şi/sau cifre (gândite ca entităţi

indivizibile), logograme sau diverse simboluri speciale reprezentând cuvinte sau fraze. Mesajele formate pe baza unui astfel de alfabet vor fi numite *texte iniţiale* sau *texte sursă* sau *texte clare* sau *plaintexte* [6]. Ele sunt secvenţe (înşiruiri) de elemente ale alfabetului. Similar, putem presupune că textele produse prin criptarea textelor sursă sunt construite pe baza unui alt alfabet (nu neapărat identic cu cel peste care se construiesc textele sursă). Textele rezultate prin criptarea textelor sursă vor fi numite *texte criptate* sau *texte cifrate* sau *criptotexte* [7].

Definiţia 4.7.1. Un *sistem de criptare* sau *criptosistem* este un 5-uplu

$$\mathcal{S} = (\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D}),$$

unde:

(i) $\mathcal{P}$ este o mulţime finită şi nevidă numită *alfabetul plaintextelor*. Elementele ei sunt numite *simboluri/caractere plaintext*;

(ii) $\mathcal{C}$ este o mulţime finită şi nevidă numită *alfabetul criptotextelor*. Elementele ei sunt numite *simboluri/caractere criptotext*;

(iii) $\mathcal{K}$ este o mulţime finită şi nevidă numită *spaţiul cheilor de criptare*. Elementele ei sunt numite *chei* (de criptare);

(iv) $\mathcal{E}$ şi $\mathcal{D}$ sunt două mulţimi de funcţii,

$$\mathcal{E} = \{e_K : \mathcal{P} \to \mathcal{C} | K \in \mathcal{K}\}$$

şi

$$\mathcal{D} = \{d_K : \mathcal{C} \to \mathcal{P} | K \in \mathcal{K}\},$$

astfel încât pentru orice $K \in \mathcal{K}$ şi $x \in \mathcal{P}$ are loc $d_K(e_K(x)) = x$.

Plaintextele (textele ce urmează a fi criptate) se construiesc pe baza alfabetului $\mathcal{P}$, iar *criptotextele* (textele ce rezultă prin criptare) sunt elemente ale mulţimii tuturor textelor construite pe baza alfabetului $\mathcal{C}$. Pentru fiecare cheie $K \in \mathcal{K}$, funcţia e_K (d_K) se numeşte *funcţia/regula de criptare* (*decriptare*). Observăm că funcţia d_K este invers la stânga al funcţiei e_K, iar e_K este invers la dreapta al funcţiei d_K. Ca urmare, e_K este funcţie injectivă, iar d_K, surjectivă. Injectivitatea regulilor de criptare conduce imediat la $|\mathcal{P}| \leq |\mathcal{C}|$.

Moduri de operare. Criptarea unui plaintext $x_1 \cdots x_n$, unde $x_i \in \mathcal{P}$ pentru orice $1 \leq i \leq n$, se poate realiza în una din următoarele două variante numite *moduri de operare*:

[6] Terminologia de "plaintext" este de provenienţă englezească şi preferăm să o utilizăm datorită uşurinţei în exprimare.

[7] Terminologia de "criptotext" provine de la cuvântul englezesc "cryptotext".

- *modul de operare cu cheie fixă.* Se alege o cheie K şi se criptează fiecare caracter x_i prin e_K. Adică criptotextul va fi

$$e_K(x_1)\cdots e_K(x_n);$$

- *modul de operare cu cheie variabilă.* Pentru fiecare i se determină o cheie K_i şi se criptează x_i prin e_{K_i}. Adică criptotextul va fi

$$e_{K_1}(x_1)\cdots e_{K_n}(x_n).$$

Determinarea cheilor K_i se face de obicei printr-un aşa numit *generator de chei*, pornindu-se de la o cheie iniţială, un anumit şir de iniţializare şi, eventual, plaintextele x_j şi criptotextele asociate lor, pentru orice $j < i$.

Criptosistemele cărora li se asociază un generator de chei şi, deci, care se utilizează în modul de operare cu cheie variabilă, se mai numesc şi *criptosisteme cu chei şir*.

Criptosistemele dezvoltate până în prezent pot fi împărţite în două clase fundamentale:

- *criptosisteme simetrice*, caracterizate prin aceea că regula decriptare se poate determina "uşor" cunoscând regula de criptare, şi invers;

- *criptosisteme asimetrice* sau *cu chei publice*, caracterizate prin aceea că cheia este împărţită în două subchei: una publică şi una privată (secretă). Subcheia publică este utilizată de oricine doreşte să transmită mesaje criptate deţinătorului cheii. Subcheia secretă este cunoscută numai de deţinătorul cheii, fiind utilizată pentru decriptarea mesajelor primite. Determinarea subcheii secrete prin cunoaşterea cheii publice trebuie să fie o problemă intractabilă.

Adesea, în cadrul criptosistemelor simetrice, alfabetul plaintextelor este o mulţime de secvenţe de lungime $m \geq 1$ formate din elemente ale unei alte mulţimi P. Adică $\mathcal{P} \subseteq P^m$. Dacă utilizatorul îşi formează plaintextele din elemente ale mulţimii P, atunci criptarea cu un altfel de criptosistem se realizează prin împărţirea plaintextului sursă în *blocuri* de lungime m (eventual ultimul bloc este completat astfel încât să aibă lungimea m), criptându-se apoi fiecare bloc în parte. Din acest motiv, astfel de criptosisteme mai sunt numite şi *criptosisteme bloc*. Argumentul fundamental care stă la baza considerării acestora rezidă din faptul că, pentru ele, regulile de criptare fac apel intensiv la elementele constitutive ale blocului. Marea majoritate a criptosistemelor simetrice ce au utilitate practică relevantă sunt criptosisteme bloc.

Interacţiunea dintre utilizatorii legali şi cei ilegali ai unui criptosistem poate fi reprezentată schematic ca în Figura 4.1, unde:

– A şi B sunt *utilizatori legali* ce au convenit a comunica utilizând un criptosistem a priori stabilit între ei, şi cu o anumită cheie de criptare K;

- C este *utilizator ilegal* ce încearcă a captura criptotexte ce se transmit între A şi B în ideea determinării plaintextelor corespunzătoare (sau, în cel mai bun caz, a cheii de criptare), sau ce încearcă a altera mesajele ce se transmit între A şi B.

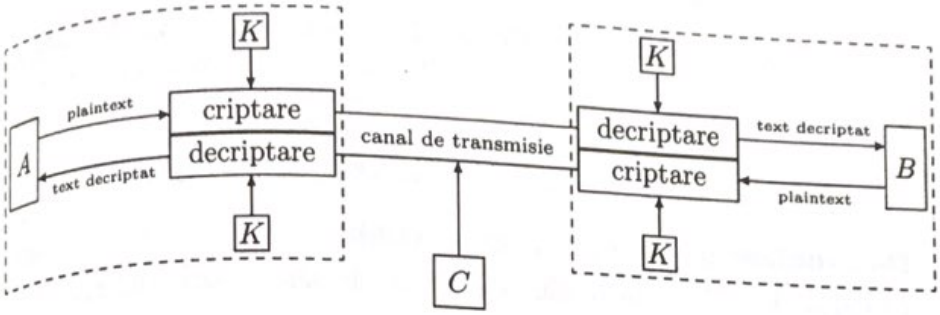

Figura 4.1: Interacţiunea dintre utilizatorii legali şi ilegali ai unui criptosistem

Construcţia criptosistemelor trebuie realizată având în vedere următoarele două deziderate fundamentale:

(1) pentru orice cheie K, funcţiile e_K şi d_K să poată fi calculate "eficient" [8] (cunoscând cheia K);

(2) determinarea cheii de criptare pe baza unor criptotexte cunoscute (eventual şi prin cunoaşterea plaintextelor corespunzătoare) să fie "imposibilă" (în sens opus sensului de la (1) de calcul eficient).

Criptanaliza este ştiinţa ce se ocupă cu studiul tehnicilor (metodelor) prin care se poate reface un plaintext pornind de la unul sau mai multe criptotexte, fără a cunoaşte, a priori, cheia de criptare. Fiecare tehnică în parte este numită *tehnică de criptanaliză* sau *tehnică de atac* sau *atac*. Dacă un criptosistem poate fi atacat cu succes printr-o anumită tehnică, atunci vom spune că el poate fi *spart* prin respectiva tehnică. Persoanele ce se ocupă cu studiul şi practicarea tehnicilor de atac se numesc *criptanalişti*. Cuplul celor două ştiinţe, criptografie şi criptanaliză, este întâlnit adesea sub terminologia de *criptologie*.

O presupunere fundamentală în criptanaliză, enunţată pentru prima dată de către Dutchman A. Kerckhoffs în secolul XIX (a se vedea [99]), constă în aceea că securitatea oferită de un criptosistem trebuie să se bazeze numai şi numai pe cheia de criptare (nu şi pe anumite detalii constructive ale acestora deoarece, mai devreme sau mai târziu, acestea pot deveni cunoscute criptanaliştilor). Altfel spus, Kerckhoffs porneşte de la ideea că, în procesul de criptanaliză, criptanaliştii au toate detaliile constructive ale criptosistemului, exceptând cheia utilizată la momentul respectiv.

[8] Prin *calculul eficient* (*uşor*) al unei funcţii vom înţelege existenţa unui algoritm determinist de complexitate timp polinomială de calcul al respectivei funcţii.

Chiar dacă în situaţii reale lucrurile nu stau în acest fel, presupunerea lui Kerckhoffs este importantă prin aceea că un criptosistem rezistent la atacuri ce iau în considerare această presupunere va fi rezistent şi la atacuri pentru care această presupunere nu este satisfăcută.

Există, în general, 4 tipuri de atacuri:

1. *atac de criptotext.* Criptanalistul are la dispoziţie un număr de criptotexte (obţinute cu aceeaşi cheie), şi scopul lui este de a determina plaintextele corespunzătoare sau cheia de criptare (în cel mai bun caz) sau un algoritm de determinare a unui (nou) plaintext pornind de la criptotextul asociat (fără a cunoaşte cheia de criptare). Schematic, aceasta poate fi exprimată ca mai jos:

   Date iniţiale: $y_1 = e_K(x_1), \ldots, y_i = e_K(x_i)$;

   Cerinţe: $x_1, \ldots, x_i$ sau K sau un algoritm de determinare a lui x_{i+1};

2. *atac de plaintext cunoscut.* Criptanalistul are la dispoziţie un număr de criptotexte (obţinute cu aceeaşi cheie), dar şi plaintextele corespunzătoare. Scopul lui este de a determina cheia de criptare sau un algoritm de determinare a unui (nou) plaintext pornind de la criptotextul asociat (fără a cunoaşte cheia de criptare). Schematic, aceasta poate fi exprimată ca mai jos:

   Date iniţiale: $(x_1, y_1 = e_K(x_1)), \ldots, (x_i, y_i = e_K(x_i))$;

   Cerinţe: K sau un algoritm de determinare a lui x_{i+1};

3. *atac de plaintext ales.* Criptanalistul are posibilitatea de a determina criptotextele unor plaintexte alese de el. Scopul lui este de a determina cheia de criptare sau un algoritm de determinare a unui (nou) plaintext pornind de la criptotextul asociat (fără a cunoaşte cheia de criptare). Schematic, aceasta poate fi exprimată ca mai jos:

   Date iniţiale: $(x_1, y_1 = e_K(x_1)), \ldots, (x_i, y_i = e_K(x_i))$, unde $x_1, \ldots, x_i$ sunt alese de criptanalist;

   Cerinţe: K sau un algoritm de determinare a lui x_{i+1} pornind de la $y_{i+1} = e_K(x_{i+1})$;

4. *atac adaptiv de plaintext ales.* Acesta este un caz particular al metodei de atac de plaintext ales. Criptanalistul are posibilitatea de a cunoaşte criptotextele asociate plaintextelor alese de el şi, mai mult, are posibilitatea de a determina noi perechi (plaintext,criptotext) cu plaintextul ales în funcţie de concluziile obţinute prin analiza perechilor (plaintext,criptotext) anterioare.

Există şi alte metode de atac, mai puţin frecvente sau eficiente. Unul dintre acestea este *atacul de criptotext ales* în care se presupune că criptanalistul are posibilitatea de a determina plaintextele corespunzătoare unor criptotexte alese de el, iar un altul constă în enumerarea şi verificarea tuturor cheilor posibile. Acest ultim atac, întâlnit

sub denumirea de *atac brute-force* sau *atac prin căutare exhaustivă a cheilor* va fi abreviat de noi prin EKS [9].

Corespondenţa literă–cifră. Marea majoritate a criptosistemelor ce sunt utilizate astăzi în practică sunt construite utilizând aparatul algebric, uzând de o anumită corespondenţă bijectivă între setul de caractere necesar construirii plaintextelor şi criptotextelor şi o anumită submulţime de numere naturale (o astfel de corespondenţă ar putea fi, de exemplu, codul EBCDIC sau ASCII cu numerele privite în sistemul zecimal). O astfel de "codificare" a setului de caractere utilizat nu trebuie gândită ca o metodă de creştere a securităţii criptosistemelor; ea nu este altceva decât un pas intermediar în aplicarea unor metode criptografice bazate în mod esenţial pe teoria numerelor.

Fără a aduce nici un prejudiciu teoriei generale, se utilizează un set redus de caractere ca cel din Figura 4.2, împreună cu *corespondenţa literă–cifră* asociată.

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12

n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

Figura 4.2: Corespondenţa literă-cifră

Astfel, se înlocuiesc 'ă' prin 'a', 'â' prin 'a', 'î' prin 'i', 'ş' prin 's' şi 'ţ' prin 't'. De asemenea, nu se face distincţie între literele mari şi mici, spaţiile sunt inserate între caractere doar pentru claritate, nu se folosesc semne de punctuaţie, caractere matematice sau de orice alt tip. Aceasta se bazează pe faptul că orice limbaj este dependent de context în mare măsură, sensul unei propoziţii fiind clar chiar prin convenţiile făcute mai sus.

Dacă $x = x_1 \cdots x_n$ este un text de lungime $n \geq 1$ peste alfabetul considerat mai sus, atunci prin *deplasarea/shiftarea* lui x cu k poziţii, unde $0 \leq k \leq 25$, înţelegem textul obţinut din x prin înlocuirea fiecărei litere x_i cu litera de pe poziţia $(k_i + k)$ *mod* 26, unde k_i este poziţia literei x_i, pentru orice i.

Criptosisteme cu chei publice. O caracteristică esenţială a criptosistemelor cu chei simetrice este aceea că cheia de criptare trebuie să fie secretă (cunoscută numai de utilizatorii legali ai criptosistemului). Această cheie furnizează atât regula de criptare cât şi cea de decriptare, fiecare din aceste reguli fiind algoritmic calculabilă în timp "eficient" (un astfel de criptosistem va fi prezentat în Secţiunea 5.6). Transmiterea cheii între utilizatorii legali trebuie realizată folosind un canal de transmisie sigur, ceea ce în practică poate constitui o problemă majoră (interceptarea cheii de către un utilizator ilegal compromite criptosistemul în totalitate).

[9] De la *Exhaustive Key Search*.

Opus ideii de cheie secretă se află conceptul de *cheie asimetrică*. Astfel de chei sunt împărţite în două subchei, una publică şi una privată (secretă). Subcheia publică este publică, putând fi utilizată de oricine pentru criptare de mesaje. Subcheia secretă este cunoscută numai de deţinătorul cheii, fiind utilizată pentru decriptare. Cunoaşterea subcheii publice nu trebuie să permită determinarea uşoară a subcheii secrete. Criptosistemele bazate pe astfel de chei se numesc *criptosisteme asimetrice* sau *criptosisteme cu chei publice*. Ideea realizării unor astfel de criptosisteme a fost prezentată în 1976 de către W. Diffie şi M.E. Hellman [45] la o Conferinţă Naţională de Informatică în SUA (a se vedea şi [46]) [10]. Primul criptosistem de acest tip a fost propus de Merkle şi Hellman [126, 127, 84] [11]. Multe alte criptosisteme au fost apoi propuse, iar unele dintre ele s-au dovedit a fi nesigure. Altele asigură securitate dar sunt inutilizabile din punct de vedere practic, iar altele ori se bazează pe chei extrem de lungi, ori criptotextul produs este foarte mare în raport cu plaintextul. Puţine dintre ele s-au dovedit a fi atât sigure cât şi practic utilizabile. Dintre acestea menţionăm criptosistemul RSA, criptosistemul Rabin, criptosistemul ElGamal, criptosistemul Chor-Rivest (din clasa criptosistemelor bazate pe problema rucsacului), criptosistemul McEliece şi criptosisteme bazate pe curbe eliptice. Primele trei pot fi utilizate şi în construcţia de semnături digitale.

Criptosistemele cu chei publice sunt predispuse atacului de plaintext cunoscut deoarece cheia este publică. Astfel, oricine interceptează un criptotext y poate încerca diverse plaintexte x pentru a obţine $e_K(x) = y$. Perechile (x, y) astfel obţinute pot atunci constitui baza unui atac de plaintext cunoscut. Ca urmare, cu cât spaţiul plaintextelor este mai mic, cu atât mai mult criptosistemul este nesigur. De fapt, discuţia de mai sus ne spune că criptosistemele cu chei publice nu pot asigura securitate necondiţionată. Deci ceea ce putem cere de la astfel de criptosisteme este doar securitatea algoritmică (ceea ce nu este puţin dacă această securitate algoritmică este "bine aleasă" adică, bazată pe probleme a căror rezolvare este algoritmic "foarte dificilă" la momentul actual).

4.7.2. Criptosistemul RSA

Criptosistemul RSA a fost propus de Ronald Rivest, Adi Shamir şi Leonard Adleman în 1977 [165], luându- si denumirea de la numele celor care l-au propus. Dintre toate criptosistemele cu chei publice propuse până în prezent, RSA s-a dovedit a fi cel mai uşor de implementat şi utilizat. El se bazează pe "dificultatea" factorizării numerelor foarte mari (a se vedea discuţia de la sfârşitul Secţiunii 2.7.2).

[10] Independent, această idee apare şi la R. Merkle dar, datorită unor probleme editoriale, prima lucrare a acestuia apare abia în 1978 [125].

[11] Şi în acest caz, lucrarea lui Merkle şi Hellman apare în 1978, ceea ce face ca criptosistemul RSA, care a fost publicat în 1977, să fie considerat de unii autori ca fiind primul astfel de criptosistem.

4.7.2.1. Descrierea criptosistemului

Criptosistemul RSA folosește aritmetica modulară în $\mathbf{Z}_n$, unde n este produsul a două numere prime distincte p și q.

Descrierea criptosistemului

- fie p și q două numere prime distincte și fie $n = pq$;
- $\mathcal{P} = \mathcal{C} = \mathbf{Z}_n$;
- $\mathcal{K} = \{(n, p, q, e, d) | e \in \mathbf{Z}^*_{\phi(n)} \ \wedge \ ed \equiv 1 \ mod \ \phi(n)\}$;
- pentru orice cheie $K = (n, p, q, e, d) \in \mathcal{K}$ și $x, y \in \mathbf{Z}_n$,

$$e_K(x) = x^e \ mod \ n \ \text{ și } \ d_K(y) = y^d \ mod \ n;$$

- pentru orice cheie $K = (n, p, q, e, d) \in \mathcal{K}$, (n, e) este cheia publică, iar (p, q, d) este cheia secretă.

Arătăm că pentru orice cheie $K = (n, p, q, e, d)$, d_K este invers la stânga a funcție e_K, adică $x^{ed} \equiv x \ mod \ n$, pentru orice $x \in \mathbf{Z}_n$. Avem de analizat două cazuri:

(1) $x \in \mathbf{Z}^*_n$. Atunci, $x^{ed} \equiv x \ mod \ n$ deoarece $ed \equiv 1 \ mod \ \phi(n)$;

(2) $x \in \mathbf{Z}_n - \mathbf{Z}^*_n$. Fie $t \in \mathbf{Z}$ astfel încât $ed - 1 = t\phi(n)$. Dacă $x = 0$, atunci are loc $x^{ed} \equiv x \ mod \ n$. Presupunem deci că $x \neq 0$. Atunci, deoarece x și n nu sunt prime între ele, $(x, n) \geq 2$. Cum $n = pq$ și $x < n$, deducem că $(x, n) = p$ sau $(x, n) = q$. Să presupunem că $(x, n) = p$ (celălalt caz este complet similar acestuia). Atunci q nu va divide x și, în baza teoremei lui Fermat, deducem că are loc

$$x^{q-1} \equiv 1 \ mod \ q$$

de la care urmează $x^{t\phi(n)} \equiv 1 \ mod \ q$. Combinând aceasta cu $x \equiv x \ mod \ p$ și cu faptul că p și q sunt prime între ele, deducem

$$x^{t\phi(n)+1} \equiv x \ mod \ pq.$$

Adică $x^{ed} \equiv x \ mod \ n$.

Ca urmare, structura definită mai sus este un criptosistem.

Presupunem acum că două părți A și B doresc să schimbe mesaje criptându-le cu RSA cu parametrii p, q și n (p și q cunoscuți doar de A și B, iar n public). Pentru aceasta, A alege o pereche de numere (e_A, d_A) ca în algoritm, face public e_A (cheia lui publică) și reține secret d_A (cheia lui secretă). Similar, B alege (e_B, d_B), face public e_B și reține secret d_B. Presupunem acum că A dorește a cripta și transmite un mesaj α către B. Acest mesaj se codifică numeric (folosind, de exemplu, corespondența deja

adoptată) după care şirul numeric obţinut se împarte în blocuri de cifre de dimensiune egală, astfel încât fiecare bloc să poată fi considerat ca un număr din $\mathbf{Z}_n$ (eventual, ultimul bloc se completează cu zerouri la dreapta pentru a avea aceeaşi lungime ca şi celelate blocuri). Fie $x = x_1 \cdots x_m$ această descompunere. Se calculează apoi

$$y_i = x_i^e \ mod \ n,$$

pentru orice $1 \leq i \leq m$. Secvenţa $y = y_1 \cdots y_m$ reprezintă codul numeric al criptotextului.

Decriptarea secvenţei y cere determinarea subsecvenţelor $y_1, \ldots, y_m$ (ca mai sus). Aceasta poate constitui o problemă serioasă pentru B, ca urmare a faptului că aceste subsecvenţe pot avea lungimi diferite iar B nu va şti cum să le separe. Această problemă poate fi depăşită dacă, înainte de a fi trimise, secvenţele $y_1, \ldots, y_m$ sunt completate la stânga cu zero-uri astfel încât toate să aibă aceeaşi lungime ca şi blocurile x_i. Fie $y'_1, \ldots, y'_m$ aceste noi blocuri. Atunci, decriptarea decurge calculând

$$x_i = (y'_i)^d \ mod \ n,$$

pentru orice $1 \leq i \leq m$.

Exemplul 4.7.2.1. Fie $p = 101$ şi $q = 113$ două numere prime. Atunci

$$n = 11413 \ \text{şi} \ \phi(n) = 11200.$$

Deoarece $\phi(n) = 2^6 5^2 7$, urmează că e poate fi ales ca fiind orice număr mai mic strict decât n ce nu este divizibil prin 2, 5 sau 7. Fie $e = 3533$. Atunci, cu ajutorul algoritmului lui Euclid determinăm $d = e^{-1} \ mod \ 11200 = 6597$.

Numerele $n = 11413$ şi $e = 3533$ sunt publice; p, q şi d sunt secrete. Criptarea textului 9726 produce criptotextul 5761 deoarece

$$9726^{3533} \ mod \ 11413 = 5761.$$

Decriptarea lui 5761 conduce, evident, la 9726.

Vom discuta în cele ce urmează câteva aspecte legate de implementarea criptosistemului RSA. Utilizarea acestui criptosistem în practică necesită parcurgerea următorilor paşi:

- alegerea a două numere prime p şi q (este necesar ca aceste numere să fie mari, de cel puţin 100 de cifre fiecare, pentru a asigura securitate criptosistemului);
- alegerea unui număr $e \in \mathbf{Z}^*_{\phi(n)}$ şi determinarea inversului modulo $\phi(n)$ al acestuia, fie el d.

Este recomandabil ca numerele p şi q să aibă lungimi apropiate (dacă nu chiar egale). Alegerea lui e constituie un alt punct cheie atât pentru asigurarea securităţii criptosistemului, cât şi pentru eficienţa implementării. Se constată că valori mici

ale lui e asigură criptare rapidă şi, totodată, dă posibilitatea unui factor mare de decriptare d, ceea ce măreşte securitatea criptosistemului (asupra acestui aspect vom reveni). Cele mai utilizate valori pentru e sunt 3, 17 şi 65537 ($2^{16}+1$) (reprezentarea binară a lui 65537 are doar doi biţi 1, ceea ce ne spune că se vor face doar 16 ridicări la puterea a 2-a şi o înmulţire).

Algoritmii ce implementează regulile e_K şi d_K sunt eficienţi, de complexitate timp $\mathcal{O}((\log n)^3)$.

Generarea numerelor prime p şi q, de 100 de cifre să spunem, ar ridica următoarea problemă care, aparent, ar fi cât se poate de serioasă: nu epuizăm, într-o perioadă de utilizare a unui astfel de criptosistem, toate numerele prime de 100 de cifre? Răspunsul este cât se poate de simplu dacă facem apel la Teorema numerelor prime. În baza ei putem spune că există proximativ $4 \cdot 10^{97}$ numere prime cu 100 de cifre, număr ce depăşeşte numărul de atomi din universul vizibil (a se vedea Secţiunea 2.1). Ca urmare, nu trebuie să ne fie teamă că am "epuiza" toate numerele prime într-o perioadă, fie ea oricât de lungă, de utilizare a criptosistemului RSA.

Testarea primalităţii unui număr p se poate face prin algoritmi probabilişti de complexitate $\mathcal{O}((\log p)^3)$ (de exemplu, algorithmul Miller-Rabin). Dezavantajul unui algoritm probabilist constă în aceea că el furnizează răspuns corect dacă numărul este prim, dar poate furniza răspuns eronat dacă numărul nu este prim (adică el ne poate spune că numărul este prim, chiar dacă acesta nu este). Iterarea unui algoritm probabilist peste o aceeaşi intrare (acelaşi număr pentru care se testează primalitatea) de un număr de ori şi obţinerea de fiecare dată a unui răspuns de genul "numărul este prim" face ca probabilitatea de răspuns eronat să scadă dramatic. De exemplu, în cazul algoritmului Miller-Rabin, iterarea de 100 de ori a acestuia poate conduce la o probabilitate de răspuns eronat de cel mult $1/4^{100}$. În 2002, Agrawal, Kayal şi Saxena au arătat că problema primalităţii poate fi rezolvată prin algoritmi determinişti de complexitate timp polinomială (lucrarea acestora a fost publicată în 2004 [1]). Algoritmii determinişti de complexitate timp polinomială cunoscuţi la momentul actual au complexitate destul de ridicată, $\mathcal{O}((\log p)^{10.5})$, ceea ce face ca tot algoritmii probabilişti să fie preferaţi deocamdată.

Generarea unui număr prim se face în două etape: se generează aleator un număr, după care se verifică primalitatea acestuia. O problemă ce se ridică relativ la generarea aleatoare a numerelor este următoarea: câte numere mari trebuie generate aleator pentru a depista cel puţin un număr prim? Pentru a răspunde acestei întrebări reamintim că Teorema numerelor prime afirmă că pentru orice număr natural m există aproximativ $m/ln(m)$ numere prime mai mici decât m. Aceasta înseamnă că, dacă p a fost generat aleator, probabilitatea ca el să fie prim este $1/ln(p)$. În cazul în care p este un număr de 512 biţi, atunci probabilitatea de mai sus este aproximativ $1/177$. Adică, în medie, generarea a 177 de numere mari va include şi un număr prim. Dacă însă cerem încă de la început ca numerele generate să fie impare, atunci probabilitatea de mai sus se dublează.

O implementare a acestei metode pe o maşină SPARC II a condus la depistarea unor numere prime de 256 de biţi în 2.8 secunde, a unor numere prime de 512 de biţi în 24 secunde, a unor numere prime de 768 de biţi în 2 minute, şi a unor numere

prime de 1024 de biţi în 5.1 minute [176].

4.7.2.2. Criptanaliză RSA

Trebuie să remarcăm că dacă se cunoaşte descompunerea lui n în factori primi, $n = pq$, criptosistemul RSA este compromis (în ipoteza că această descompunere este cunoscută de o persoană neautorizată). În adevăr, cunoscând p şi q se poate determina uşor $\phi(n)$, şi atunci cunoaşterea cheii publice e conduce la determinarea imediată a cheii secrete d (în timp $\mathcal{O}((\log n)^3)$). Calcularea lui $\phi(n)$ în timp polinomial determinist cunoscând doar n, are aceleaşi consecinţe. Calculul lui $\phi(n)$ în timp eficient face apel tot la factorizarea lui n. Ca urmare, putem spune că criptosistemul RSA îşi bazează securitatea pe problema factorizării pentru care, la momentul actual, nu se cunosc metode eficiente (dacă numerele prime p şi q sunt alese aleator şi suficient de mari).

Vom prezenta în continuare 3 modalităţi de a ataca criptosistemul RSA, fără a cunoaşte factorizarea lui n. Fiecare din aceste atacuri speculează utilizare "greşită" a criptosistemului sau alegere nepotrivită a parametrilor.

Atacul lui Davida a fost propus în [39], el exploatând utilizare cu greşeli a criptosistemului.

Presupunem că A trimite un mesaj x lui B folosind cheia publică e_B. Mai presupunem că C intercepteză mesajul transmis, $y = x^{e_B} \ mod \ n$, şi multiplică y cu z, unde z este ales astfel încât există $z^{-1} \ mod \ n$, şi trimite $y' = yz^{e_B} \ mod \ n$ lui B. Decriptând acest mesaj, B va găsi $x' = y'^{d_B} \ mod \ n$ care, foarte probabil, este un plaintext fără înţeles. Ca urmare, el va renunţa la acest mesaj. Dacă C va putea obţine x', atunci el va putea determina x prin

$$x = x'z^{-1} \ mod \ n.$$

Atacul lui Lenstra a fost propus în [111], el exploatând, ca şi atacul lui Davida, utilizare cu greşeli a criptosistemului.

Decriptarea unui mesaj criptat cu RSA poate fi realizată eficient dacă, dat $y = x^e \ mod \ n$, se calculează

$$x_p = y^{d \ mod \ p-1} \ mod \ p$$

şi

$$x_q = y^{d \ mod \ q-1} \ mod \ q.$$

De la acestea, x se obţine cu ajutorul Teoremei chineze a resturilor ca fiind unica soluţie modulo $n = pq$ a sistemului

$$\begin{cases} x & \equiv & x_p \ mod \ p \\ x & \equiv & x_q \ mod \ q \end{cases}$$

Să presupunem că x_p a fost calculat corect dar x_q a fost calculat greşit. Ca urmare, rezolvarea sistemului de mai sus va conduce la un x' diferit de x şi, privit ca plaintext,

fără înţeles. Dacă acest x' este obţinut de C, atunci se poate determina în timp eficient p prin

$$p = ((x'^{e_B} - y) \ mod \ n, n).$$

Ca urmare, criptosistemul este complet neutralizat de C.

Atac de exponent secret mic (atacul lui Wiener). Atacul pe care îl prezentăm aici este datorat lui M. Wiener [212]. El exploatează alegerea unui parametru e care produce o cheie secretă d "prea mică".

Teorema 4.7.2.1. Fie p şi q numere prime astfel încât $q < p < 2q$, $n = pq$ şi $e, d \in \mathbf{Z}^*_{\phi(n)}$ astfel încât $ed \equiv 1 \ mod \ \phi(n)$. Dacă $d < 1/3\sqrt[4]{n}$, atunci d se poate determina în timp polinomial determinist în raport cu $log\, n$ cunoscând doar n şi e.

Demonstraţie. Relaţia $e, d \in \mathbf{Z}^*_{\phi(n)}$ conduce la existenţa unui număr natural k astfel încât $ed - 1 = k\phi(n)$ sau, echivalent,

$$\left|\frac{e}{\phi(n)} - \frac{k}{d}\right| = \frac{1}{d\phi(n)}. \qquad (1)$$

Mai mult, $ed - k\phi(n) = 1$ ne spune că $(k, d) = 1$, deci fracţia k/d este ireductibilă.

Ca urmare, determinarea lui d poate fi redusă la determinarea unor fracţii ireductibile k/d care să verifice (1). Vom arăta că orice fracţie ireductibilă k/d ce verifică (1) va verifica şi

$$\left|\frac{e}{n} - \frac{k}{d}\right| < \frac{1}{2d^2}. \qquad (2)$$

În plus, determinarea fracţiilor k/d ce verifică (2) se poate face în timp polinomial în raport cu $log\, n$.

Să presupunem deci că k/d verifică (1). Stabilim întâi următoarele relaţii:

$$\begin{aligned} n - \phi(n) &= p + q - 1 \\ &< 3q - 1 && \text{(de la } p < 2q) \\ &< 3q \\ &< 3\sqrt{n} && \text{(de la } q < p \text{ şi } n = pq) \end{aligned}$$

şi

$$\begin{aligned} k\phi(n) &= ed - 1 \\ &< ed \\ &< d\phi(n) && \text{(de la } e < \phi(n)) \\ &< 1/3\sqrt[4]{n}\,\phi(n), && \text{(conform ipotezei)} \end{aligned}$$

care conduce la $k < 1/3\sqrt[4]{n}$.

Acum obţinem:

$$\begin{aligned}
|e/n - k/d| &= |ed - kn|/(nd) \\
&= |1 + k\phi(n) - kn|/(nd) \\
&= |k(n - \phi(n)) - 1|/(nd) \\
&< (k(n - \phi(n)))/(nd) \\
&< (3k\sqrt{n})(nd) \\
&< (3k)(d\sqrt{n}) \\
&< (\sqrt[4]{n})(d\sqrt{n}) \\
&= 1/(d\sqrt[4]{n}) \\
&< 1/(3d^2) \qquad \text{(de la } \sqrt[4]{n} > 3d) \\
&< 1/(2d^2).
\end{aligned}$$

Deci k/d verifică.(2).

Fracţiile k/d ce verifică (2) sunt exact convergentele fracţiei continue finite simple asociate lui e/n [80]. Acestea se obţin pe baza câturilor succesive ale împărţirii lui e la n prin algoritmul lui Euclid (a se vedea Secţiunea 2.2). Există $log\, n$ astfel de convergente ce pot fi determinate în timp polinomial în raport cu $log\, n$. Verificarea faptului că una din aceste convergente este exact fracţia k/d ce verifică (1) se face tot în timp polinomial în raport cu $log\, n$. Deci d se poate determina în timp polinomial în raport cu $log\, n$. □

Cu valori de 512 biţi pentru p şi q, va rezulta n de 1024 biţi. Ca urmare, pentru a contracara atacul lui Wiener, d trebuie să fie de cel puţin 256 biţi.

Cele prezentate de noi în această secţiune nu au dorit altceva decât să scoată în evidenţă subtilitatea atacurilor ce se pot monta asupra criptosistemelor şi, de ce nu, să atragă interesul cititorului spre acest domeniu extrem de util şi interesant.

4.7.3. Semnături digitale

O altă aplicaţie importantă a problemelor algoritmice ce apar în cadrul grupului $\mathbf{Z}_m^*$ o constituie semnătura digitală.

4.7.3.1. Introducere

Semnătura are scopul de a certifica originalitatea datelor care se transmit între diverse părţi. Frecvent, semnăm cecuri, scrisori, contracte; originalitatea semnăturii noastre conferă textului în cauză caracterul de original. În general, o semnătură trebuie să satisfacă următoarele cerinţe:

- să fie autentică (produsul original al semnatarului);

- să fie nefalsificabilă;
- să nu poate fi reutilizată (odată folosită pentru un document, să nu poată fi transferată pe alt document prin diverse tehnici);
- să nu poată fi repudiată (renegată) de semnatarul ei.

În realitate, nici una din cerinţele de mai sus nu este complet satisfăcută. Există diverse metode de verificare a originalităţii unei semnături (analize grafologice etc.) dar care nu conferă garanţii complete.

Necesitatea utilizării semnăturilor este cât se poate de clară pentru oricine dintre noi. Să presupunem că A şi B sunt două părţi ce doresc să comunice, schimbând între ele diverse mesaje (documente, acte, scrisori etc.). Există cel puţin două aspecte fundamentale care trebuiesc luate în consideraţie:

1. dacă B primeşte un mesaj x de la A, atunci B doreşte să aibă garanţia că x este, în adevăr, mesajul original trimis de A (acest mesaj nu a fost schimbat pe parcurs de o terţă persoană C sau, chiar de către A – A comunică lui B că îi va trimite mesajul x dar, în fapt, el trimite un alt mesaj invocând, la nevoie, posibilitatea alterării mesajului pe canalul de transmisie);
2. dacă A trimite mesajul x către B, atunci A doreşte să aibă garanţia că mesajul x este cel recepţionat de B (mesajul nu a fost schimbat pe parcurs de o terţă persoană C sau, chiar de către B).

Într-un astfel de context, utilizarea semnăturii (personale) constituie soluţia problemei. A va semna suplimentar mesajul x prin determinarea, după o procedură de semnare secretă (personală), a entităţii $sig(x)$ (semnătură grafică, amprentă digitală etc.). Când B va recepţiona cuplul $(x, sig(x))$, acesta va trebui să verifice, cu o procedură publică $ver(x, sig(x))$, autenticitatea semnăturii $sig(x)$ asupra mesajului x. Interpunerea lui C între A şi B, fără a cunoaşte procedura de semnare, trebuie să fie ineficientă.

Discuţia purtată până acum poate fi bine descrisă prin intermediul Figurii 4.3.

$$\mathrm{A} \xrightarrow{\;x\;} \mathrm{B}$$

$$\mathrm{A} \xrightarrow{\;x\;} \mathrm{C} \xrightarrow{\;x'\;} \mathrm{B}$$

$$\mathrm{A} \xrightarrow{\;x,\ sig_A(x)\;} \mathrm{C} \xrightarrow{\;x',\ ?\;} \mathrm{B}$$

Figura 4.3: Schema de semnare cu interpunerea unei terţe părţi

În 1976, Diffie şi Hellman [46], odată cu lansarea ideii de cheie publică propune utilizarea semnăturilor şi în cazul transmisiei datelor prin intermediul calculatorului

(mesaje, programe etc.). Cerinţele pe care trebuie să le satisfacă o semnătură, cât şi precauţiile (măsurile de siguranţă) pe care trebuie să şi le ia părţile implicate în comunicare, discutate mai sus, trebuie să rămână valabile şi în acest caz. Spre deosebire de semnătura uzuală a fiecăruia, ce este în linii mari aceeaşi pe orice document, semnătura digitală variază de la document la document.

Definiţia 4.7.3.1. O *schemă de semnare* sau *semnătură digitală* este un 5-uplu

$$Sig = (\mathcal{P}, \mathcal{S}, \mathcal{K}, \mathcal{A}_s, \mathcal{A}_v),$$

unde:

(1) $\mathcal{P}$ este o mulţime finită şi nevidă ale cărei elemente sunt numite *mesaje*;

(2) $\mathcal{S}$ este o mulţime finită şi nevidă ale cărei elemente sunt numite *semnături*;

(3) $\mathcal{K}$ este o mulţime finită şi nevidă ale cărei elemente sunt numite *chei de semnare*;

(4) $\mathcal{A}_s = \{sig_K : \mathcal{P} \to \mathcal{S} | K \in \mathcal{K}\}$ şi $\mathcal{A}_v = \{ver_K : \mathcal{P} \times \mathcal{S} \to \{0,1\} | K \in \mathcal{K}\}$ sunt mulţimi $\mathcal{K}$-indexate ale căror elemente sunt numite *reguli/algoritmi de semnare* şi, respectiv, *reguli/algoritmi de verificare*, astfel încât

$$ver_K(x, y) = 1 \quad \Leftrightarrow \quad sig_K(x) = y,$$

pentru orice $x \in \mathcal{P}$, $y \in \mathcal{S}$ şi $K \in \mathcal{K}$.

Utilizarea unei scheme de semnare de către două părţi A şi B decurge astfel:

- *semnare.* A decide asupra mesajului pe care îl are de transmis lui B, fie acesta x, alege o cheie K de semnare, semnează obţinând $sig_K(x)$, şi transmite lui B cuplul $(x, sig_K(x))$;
- *verificare.* B primeşte un cuplu $(x, sig_K(x))$ şi acceptă semnătura lui A dacă şi numai dacă $ver_K(x, sig_K(x)) = 1$.

O cerinţă naturală asupra schemelor de semnare digitală constă în aceea că semnarea şi verificarea trebuie să se facă "uşor" (în timp polinomial determinist). Algoritmii de semnare trebuie să fie secreţi, iar cei de verificare, publici. Interceptarea unui mesaj şi a unei semnături asociate nu trebuie să permită determinarea algoritmului de semnare şi nici falsificarea semnăturii prin alte mijloace (semnarea corectă a altor mesaje fără a cunoaşte algoritmul de semnare).

Semnăturile digitale nu pot asigura securitate necondiţionată pentru simplul motiv că verificarea tuturor posibilităţilor (ce sunt în număr finit) conduce la determinarea algoritmului de semnare utilizat.

Semnăturile digitale pot fi utilizate în conjuncţie cu metodele de criptare cu chei publice astfel. Să presupunem că A doreşte să transmită lui B un mesaj x. A îşi alege un algoritm (propriu) de semnare sig_A, semnează mesajul, fie $y = sig_A(x)$, şi

criptează cuplul (x, y) folosind cheia publică e_B a lui B. Rezultatul, $z = e_B(x, y)$, este trimis lui B.

Evident, A poate cripta întâi pe x prin $e_B(x)$ şi apoi semna rezultatul. Dar, în acest caz, un utilizator ilegal C care interceptează cuplul

$$(e_B(x), sig_A(e_B(x)))$$

poate înlocui semnătura lui A, $sig_A(e_B(x))$, fără a cunoaşte mesajul original x, prin semnătura lui proprie $sig_C(e_B(x))$ şi trimite lui B cuplul

$$(e_B(x), sig_C(e_B(x))).$$

În urma recepţionării acestui cuplu, B va considera mesajul ca venind din partea lui C şi nu a lui A (mesajul poartă semnătura lui C). O astfel de situaţie poartă denumirea de *impersonificare*. Pentru a o evita, se recomandă întâi semnarea mesajului şi apoi criptarea cuplului (mesaj, semnătură).

Adesea cheia K utilizată pentru semnare conţine un parametru k, numit *parametru de securitate*, care, la schimbarea cheii, se schimbă doar el. În acest caz este de preferat de păstrat restul cheii drept cheie şi de gândit parametrul de securitate ca fiind un parametru auxiliar. Altfel spus, este de preferat ca procedura de semnare să fie o funcţie de două variabile, $sig_K(x, k)$. Evident, aceasta nu modifică cu absolut nimic strategia generală.

4.7.3.2. Semnătura ElGamal

Semnătura ElGamal a fost propusă în 1985 [50]. Această schemă este nedeterministă în sensul că pentru un mesaj x pot exista mai multe semnături valide; algoritmul de verificare trebuie să fie capabil de a accepta oricare din acestea ca semnături autentice.

Ideea de bază în cadrul acestei semnături este de a semna un mesaj $x \in \mathbf{Z}_p^*$, unde p este un număr prim, printr-o pereche $(\gamma, \delta) \in \mathbf{Z}_p^* \times \mathbf{Z}_{p-1}$ astfel încât x este combinaţie liniară de γ şi δ cu doi parametri secreţi a şi k,

$$x = (a\gamma + k\delta) \ mod \ (p-1).$$

Această combinaţie este aleasă modulo $p-1$ deoarece verificarea semnăturii se va face prin intermediul echivalenţei

$$u \equiv v \ mod \ (p-1) \quad \Leftrightarrow \quad \alpha^u \equiv \alpha^v \ mod \ p,$$

pentru orice rădăcină primitivă α a lui $\mathbf{Z}_p^*$.

Descrierea semnăturii:

- fie p un număr prim şi α o rădăcină primitivă modulo p;
- $\mathcal{P} = \mathbf{Z}_p^*$;